

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    /* malloc */
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc values: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);

    printf("\n");

    /* calloc */
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc values: ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);

    printf("\n");

    /* realloc */
    int *temp = (int *)realloc(a, 10 * sizeof(int));

    if (temp == NULL)
    {
```

```

    {
        printf("realloc failed\n");
        free(a);
        free(b);
        return 1;
    }

    a = temp;

    for (i = 5; i < 10; i++)
        a[i] = i + 1;

    printf("realloc values: ");
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);

    printf("\n");

    /* Free malloc/realloc memory */
    free(a);

    /*
     * b is intentionally NOT freed.
     * This creates a memory leak for Valgrind.
     */

    return 0;
}

```

Output:

```

malloc values: 1 2 3 4 5
calloc values: 0 0 0 0 0
realloc values: 1 2 3 4 5 6 7 8 9 10

```

```
nishanth@LAPTOP-6Q3ON07H:~/Practical8$ valgrind --leak-check=full ./program1
==564== Memcheck, a memory error detector
==564== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==564== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==564== Command: ./program1
==564==
malloc values: 1 2 3 4 5
calloc values: 0 0 0 0 0
realloc values: 1 2 3 4 5 6 7 8 9 10
==564==
==564== HEAP SUMMARY:
==564==   in use at exit: 20 bytes in 1 blocks
==564==   total heap usage: 4 allocs, 3 frees, 1,104 bytes allocated
==564==
==564== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==564==    at 0x4857E43: calloc (vg_replace_malloc.c:1678)
==564==    by 0x40012DD: main (in /home/nishanth/Practical8/program1)
==564==
==564== LEAK SUMMARY:
==564==    definitely lost: 20 bytes in 1 blocks
==564==    indirectly lost: 0 bytes in 0 blocks
==564==    possibly lost: 0 bytes in 0 blocks
==564==    still reachable: 0 bytes in 0 blocks
==564==    suppressed: 0 bytes in 0 blocks
==564==
==564== For lists of detected and suppressed errors, rerun with: -s
==564== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

```

#include <sys/wait.h>

int main()
{
    int data = 100;

    printf("Before fork:\n");
    printf("Parent PID = %d\n", getpid());
    printf("Value = %d\n", data);
    printf("Address = %p\n", (void *)&data);

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");

        printf("Before modification:\n");
        printf("Child PID = %d\n", getpid());
        printf("Value = %d\n", data);
        printf("Address = %p\n", (void *)&data);

        data = 200;

        printf("\nAfter modification:\n");
        printf("Child value = %d\n", data);
        printf("Child address = %p\n", (void *)&data);
    }
    else
    {
        wait(NULL);

        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID = %d\n", getpid());
        printf("Parent value = %d\n", data);
        printf("Parent address = %p\n", (void *)&data);
    }

    return 0;
}

```

Output:

```
nishanth@LAPTOP-6Q30N07H:~/Practical8$ gcc program2.c -o program2
nishanth@LAPTOP-6Q30N07H:~/Practical8$ ./program2
Before fork:
Parent PID = 636
Value = 100
Address = 0x7ffc207b2ea0

--- CHILD PROCESS ---
Before modification:
Child PID = 637
Value = 100
Address = 0x7ffc207b2ea0

After modification:
Child value = 200
Child address = 0x7ffc207b2ea0

--- PARENT PROCESS ---
Parent PID = 636
Parent value = 100
Parent address = 0x7ffc207b2ea0
```